HW2: GAN Stability and Latent Space Exploration

Goal: Understand the practical challenges of training GANs from scratch and explore the high-dimensional latent space of a state-of-the-art generator.

Part 1: Training a DCGAN and a WGAN-GP

Using the LFW (Labeled Faces in the Wild) dataset, you will implement two different GAN architectures to observe how loss functions affect training stability.

WGAN-GP is Wasserstein GAN with Gradient Penalty. In the original **WGAN**, to enforce the continuity constraint (Lipschitz continuity), weight clipping was adopted. But that often leads to exploding or vanishing gradients. **WGAN-GP** replacesweight clipping with a **gradient penalty**: instead of forcing the weights into a box range, it adds a term to the loss function to penalize the model if the norm of its gradients deviates from 1. This provides a smoother and more stable training signal.

Task 1: Implementation and Loss Plotting.

Implement both a standard DCGAN and a WGAN-GP. Plot and submit the following curves:

- **For DCGAN:** Plot Discriminator loss and Generator loss. Comment on whether these losses reflect or correlate to the quality of generated images.
- **For WGAN:** Plot the **Wasserstein Distance** (the Critic's loss). Comment on whether this loss reflects or correlates to the quality of generated images.

Task 2: Visual Result Showing.

Use a fixed noise seed, generate a grid of 16 images at four specific training milestones after 5 epochs, 35 epochs, 65 epochs, and 100 epochs. Also compare DCGAN and WGAN.

Submission for Part 1:

- **The Loss Plots**
- **The milestone image grid** (generated 16-image grids at epochs 5, 35, 65, and 100 for both models)
- **Observations**: brief comments on the correlation between loss values and image quality for both architectures

Part 2: Exploring StyleGAN – Navigating the Latent Space

We will use a powerful pre-trained face generation model, StyleGAN2-ADA, to explore a 512-dimensional "style" space (W). Unlike standard image editors, we don't move pixels; we move through a high-dimensional map of human features. Your task is to find specific "anchor" faces and document the transformation between them.

A. **Phase 1: Iterative Sampling (The Gallery)**

Before we can interpolate, we must know what is available in the model's imagination. Generate 50 random faces, name them as Seeds 1–50, and save them into a gallery folder.

B. **Phase 2: Generate three latent morphing sequences**

1) Find a young child and an elderly person. Generate a 10-step morph between them.
2) Find one person looking to the far left and another looking to the far right. Generate a 10-step rotation morphing sequence .
3) Find one person who looks casual/sporty (e.g., messy hair, t-shirt, hat) and one who looks formal/professional (e.g., suit, glasses, styled hair). Generate the 10-step morphing between them.

Submission for Part 2:

- **The gallery of 50 generated faces.**
- **The three 10-image strip sequences**: each sequence is a single horizontal strip (concatenated by 10 images) showing the transition from Seed A to Seed B. One example given below.
- **Observations**: briefly describe how the StyleGAN handled the middle states. Does the transition feel smooth, or do certain features appear/disappear abruptly?

StyleGAN2-ADA Setup Guide:

Note: StyleGAN2-ADA is a high-performance model that uses custom C++ and CUDA kernels. If you skip the C++ compiler or use the wrong versions of Python/PyTorch, it will crash. The following steps help you install it.

Do not install this in your main coding/class environment. Instead, create a new, isolated "sandbox" environment as described below.

Step 0. Install Prerequisites.

a. **Git for Windows:** Download from **git-scm.com**.

b. **Visual Studio Community:** Download from **visualstudio.microsoft.com**.

- **Crucial:** During installation, check the box for **"Desktop development with C++"**.

c. **CUDA Toolkit 11.1**: Download the "Full Local Installer" from the NVIDIA Archive.

- Select Windows -> x86_64 -> 10 (this works for Windows 11 too) -> exe (local).
- Choose "Express Installation" and restart your computer after it finishes.

Step 1. Clone the StyleGAN2-ADA Project from GitHub.

Open your **MINGW64 (Git Bash)** window, navigate to your code folder and run:

```
git clone https://github.com/NVlabs/stylegan2-ada-pytorch.git stylegan2ada
```

then

```
cd stylegan2ada
```

Step 2. Create the Isolated Environment.

Open your Anaconda Prompt (not Git Bash for this part) and run:

```
conda create --name sg2ada python=3.7 -y
```

then

```
conda activate sg2ada
```

Step 3. Install Core Libraries.

While inside the (sg2ada) environment, run these commands in order:

pip install torch==1.7.1+cu110 torchvision==0.8.2+cu110 -f https://download.pytorch.org/whl/torch_stable.html

pip install click requests tqdm pyspng ninja imageio-ffmpeg==0.4.3

Step 4. Compiler Linking and Verification.

Link the Compiler:

Before running the code, link the environment to the Visual Studio compiler:

call "C:\Program Files\Microsoft Visual Studio\2019\Community\VC\Auxiliary\Build\vcvars64.bat"

(Note: If you installed VS 2022 or other version, change "2019" to "2022" or other in the path above.)

Verify with Face Generation

Run the generation script. This will create 3 images in a folder named “out”.

python generate.py --outdir=out --trunc=1 --seeds=1,2,3 \ --network=https://nvlabs-fi-cdn.nvidia.com/stylegan2-ada-pytorch/pretrained/ffhq.pkl